PowerShell Conference Europe

Next Up:

Marc-André Moreau

Wiesbaden

3

2

1

PowerShell Conference Europe

Implementing PSRemoting at Devolutions: Lessons Learned

Marc-André Moreau

Wiesbaden

Many thanks to our sponsors:

CTO, Devolutions

Marc-André Moreau

- Chief Technology Officer at Devolutions
- Microsoft MVP (PowerShell & Windows)
- Creator of the FreeRDP open-source project
- I love PowerShell, .NET, Rust, network protocols and reversing Windows internals ☺

About Devolutions

Canadian software company building secure remote access and privileged access management solutions

Main products

- **Devolutions RDM**— flagship product
- **Devolutions Server** — self-hosted vault & PAM
- **Devolutions Cloud** — cloud-hosted vault
- **Devolutions Gateway** — JIT access & session monitoring
- **Devolutions PowerShell Universal** — IT automation platform
- **Devolutions UniGetUI** — GUI for package managers

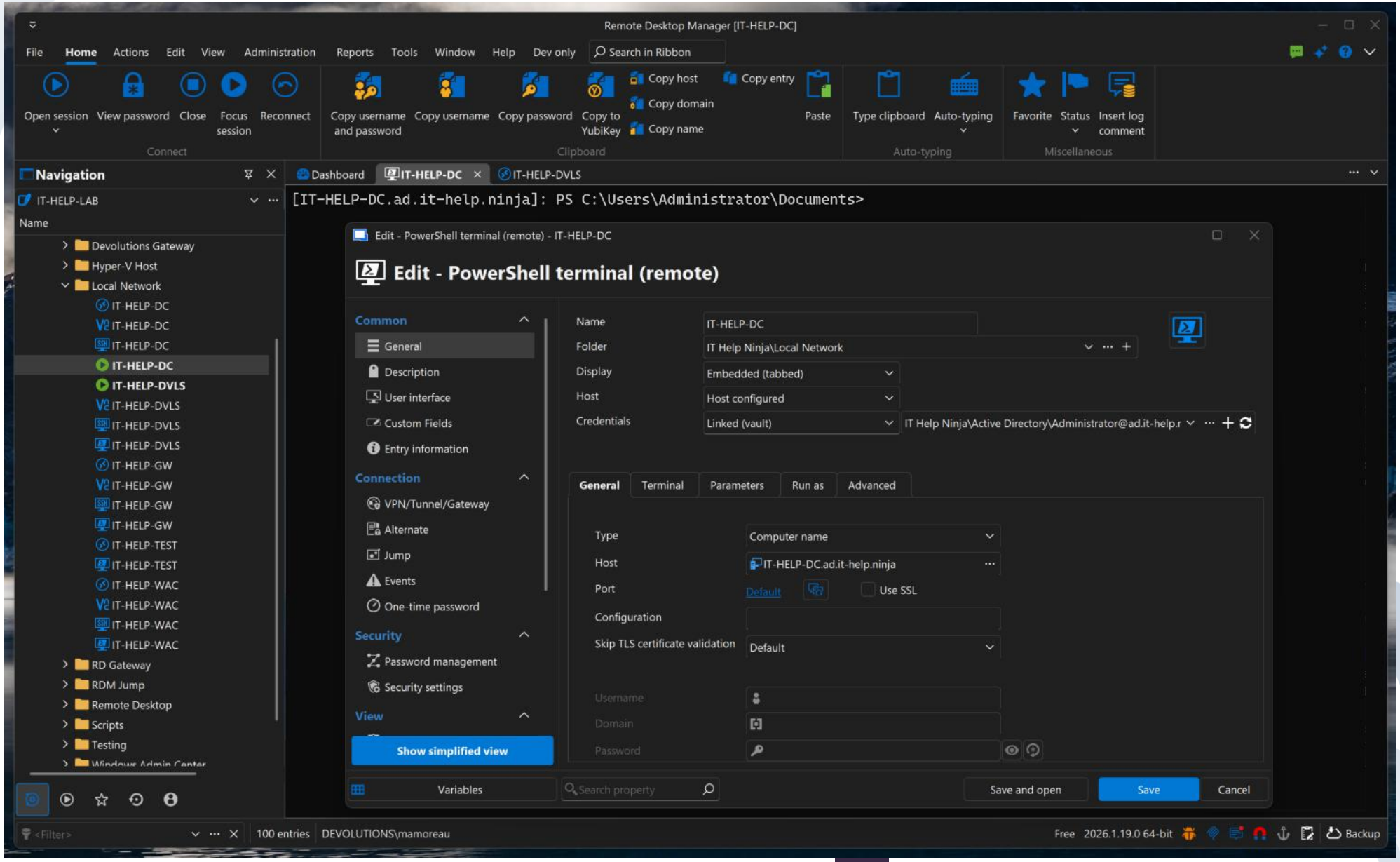
What We'll Cover

- **PSRemoting Interactive Session Launching** — how to launch an interactive PSRemoting session, embedded in RDM, and with secure credential injection
- **PSRemoting over SSH and Win32-OpenSSH** — how to launch a PSRemoting over SSH connection from RDM when Win32-OpenSSH doesn't support credential injection
- **PowerShell SDK and Local PSRemoting** — how to work around assembly conflicts within a parent .NET host process by using a local PSRemoting connection instead
- **PSRemoting in the Browser** — How to implement a true PSRemoting browser-based client that doesn't require an agent on the destination host

PSRemoting Interactive Session Launching

@awakecoding.com

Remote PowerShell in Remote Desktop Manager



Embedded PowerShell in RDM

RDM launches a new PowerShell process, reparented into an embedded tab

Classic Console Host

- Window reparenting worked, but with caveats
  - Console host appears externally for a moment before being reparented
  - Resizing doesn't automatically sync rows/cols in PowerShell

Windows Terminal

- Window reparenting is not possible with Windows Terminal
- Built our own WT distro patched to accept a parent window from RDM
- github.com/Devolutions/wt-distro

Secure Credential Injection

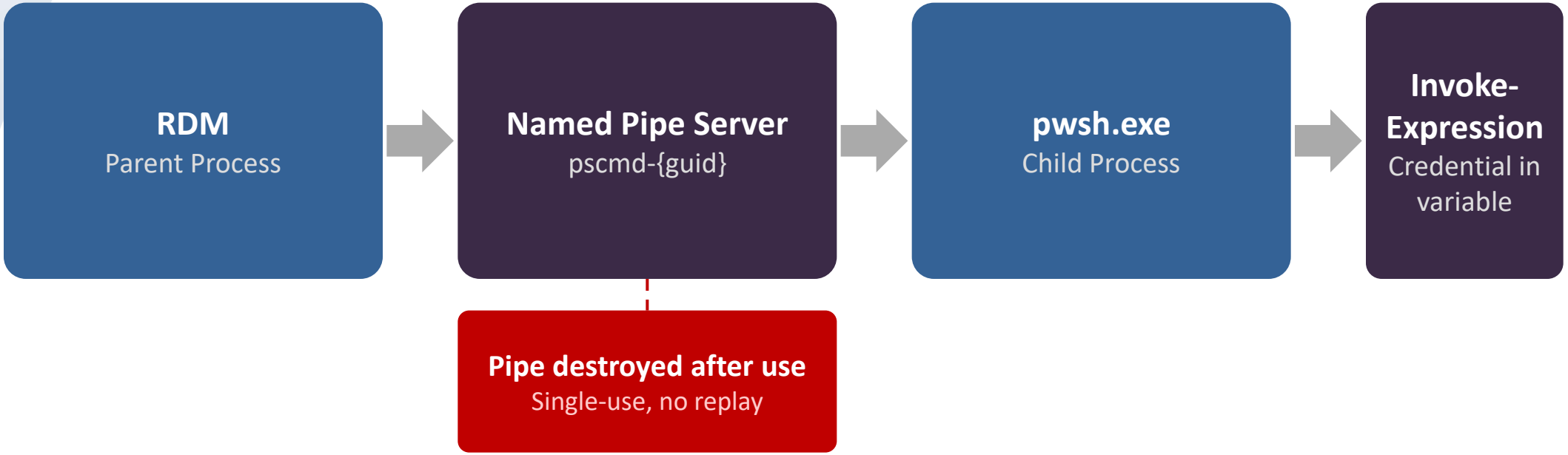
The Problem

- RDM injects Enter-PSSession with credentials at launch via pwsh.exe args
- Command-line args visible to any process on the system — credentials leaked

The Solution

- Bootstrap snippet reads the real command from a temporary named pipe server
- Enter-PSSession credentials delivered through the pipe, not the command line
- Credential reconstructed in a variable — never displayed or written to history

How PSSecureCommand Works



github.com/awakecoding/PSSecureCommand

PSRemoting over SSH and Win32-OpenSSH

@awakecoding.com

PSRemoting over SSH — Under the Hood

PSCONF.EU
Wiesbaden

Client Side

- Enter-PSSession -HostName finds the ssh executable
- Launches ssh as child process with -s powershell subsystem flag
- Hooks ssh's stdin/stdout as the MS-PSRP transport

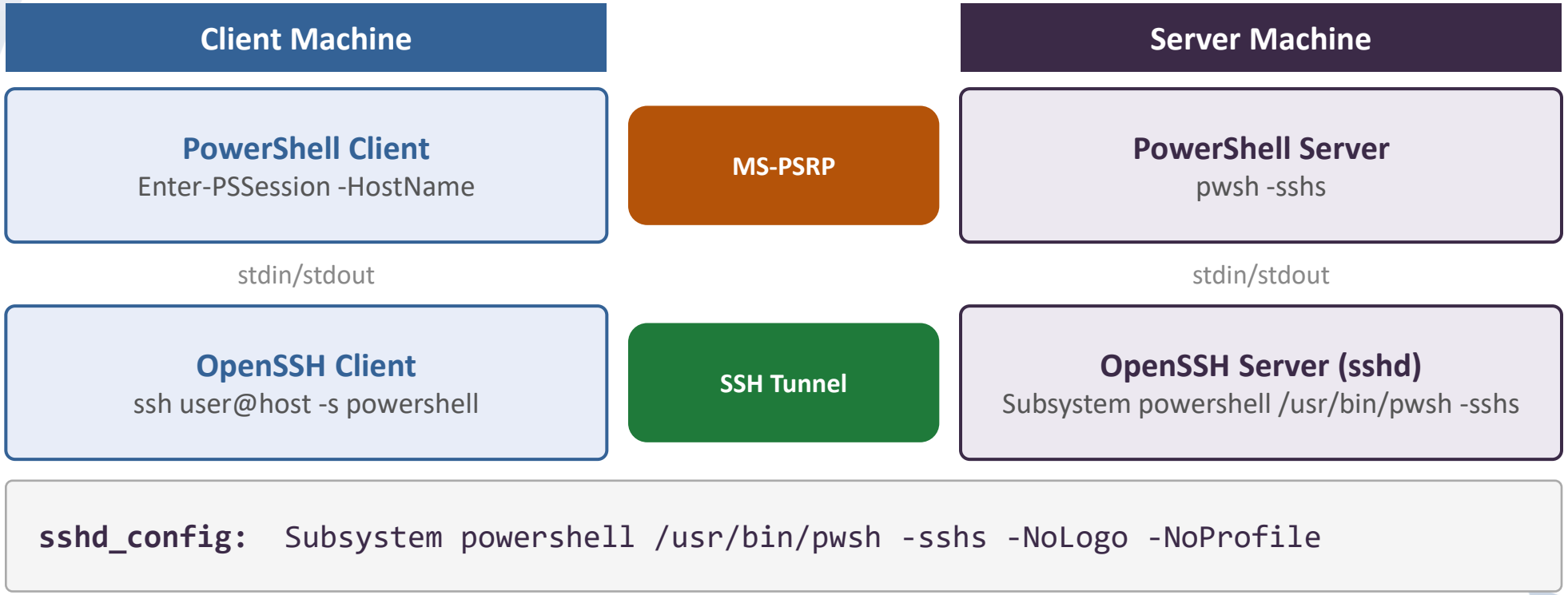
Server Side

- sshd receives the "powershell" subsystem request
- Looks up Subsystem powershell in sshd_config, launches pwsh -sshs
- Hooks pwsh stdin/stdout as the other end of the transport

Key Insight

- SSH is just a transport — it knows nothing about PowerShell
- MS-PSRP flows end-to-end between two pwsh instances over stdio

PSRemoting over SSH — Architecture

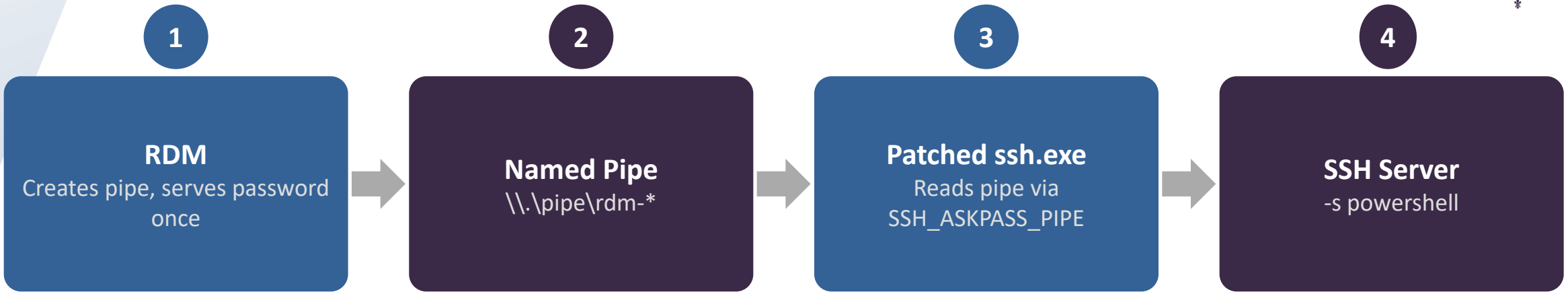



```
sshd_config:  Subsystem powershell /usr/bin/pwsh -sshs -NoLogo -NoProfile
```

SSH is just a transport pipe — MS-PSRP flows end-to-end between two pwsh instances over stdio

Patched Win32-OpenSSH for Password Injection

Windows' ssh.exe has no API to inject passwords. We ship a patched distro in RDM.



`SSH_ASKPASS_PIPE=rdm-askpass-{guid}`
Created per-connection, serves password once, then destroyed. Never written to disk.

github.com/Devolutions/openssh-distro

A Future without OpenSSH in PowerShell

PowerShell is hardcoded to expect OpenSSH

- This makes it very difficult to bring your own SSH stack with custom features
- Enter-PSSession doesn't accept –Credential for an SSH transport type

Problem: Win32-OpenSSH on Windows, vanilla OpenSSH elsewhere

- The PowerShell team can only customize Win32-OpenSSH
- OpenSSH project will likely never accept Win32-OpenSSH patches
- Conclusion: enhancements like Entra ID auth in SSH have major constraints

Solution: Pure .NET SSH client stack for PSRemoting

- Main issue beyond the .NET SSH stack is the .NET crypto stack
- Only realistic way to improve SSH features in PowerShell long term

PowerShell SDK and Local PSRemoting

@awakecoding.com

PowerShell SDK Limitations

The PowerShell SDK Bundles a Whole Copy of PowerShell

- This is extremely confusing to users that expect your .NET app to use pwsh.exe
- Bundling a whole copy of PowerShell contributes greatly to the .NET app size
- Due to the dynamic nature of PowerShell, forget about NativeAOT and trimming
- No way to import just “client” part of the PowerShell SDK without the server

Assembly Conflicts with parent .NET app

- Your .NET app likely loads assemblies that conflicts with PowerShell modules
- Workaround: run PowerShell in a separate process

Local PSRemoting via Subprocess

pwsh Subprocess in Server Mode

- Launch pwsh as a subprocess in server mode
  - Similar to how pwsh works in SSH server mode
  - Transport is stdin/stdout instead of network — no WinRM needed

No Network Auth — No Problems

- No TrustedHosts, no NTLM, no Kerberos required for local use
- Out-of-process PowerShell host without network complexity
- Assembly conflicts solved — modules load in isolated pwsh process

github.com/awakecoding/AwakeCoding.PSRemoting

Localhost WinRM: New Problems

PSCONF.EU
Wiesbaden

PowerShell 7 WinRM Registration

- Enable-PSRemoting must run from pwsh.exe, not powershell.exe
- PS7 updates break session configs — fix by unregistering and re-running Enable-PSRemoting
  awakecoding.com/posts/enable-powershell-winrm-remoting-in-powershell-7

TrustedHosts: A Misunderstood Setting

- “Trusted” hosts are actually **less** trustworthy — it’s a security exception list
- System-wide, can’t be overridden per-app — customers must modify it for DVLS
- NTLM deprecation + non-domain-joined machines = no good auth option for local WinRM
  awakecoding.com/posts/powershell-remoting-trusted-hosts-what-does-it-mean

Custom C# PSRemoting Stack

Support for local and remote MS-PSRP transport

- Local PowerShell subprocess in server mode, through standard input/output
- Named pipe transports were originally attempted, but stdin is much easier
- Remote PowerShell using WinRM, with greater control on connection sequence
- Custom C# MS-PSRP stack has none of the PowerShell SDK limitations

Used in a variety of places where the PowerShell SDK fails

- Connecting through Devolutions Gateway for Devolutions PAM
- Interactive PowerShell Remoting from RDM Android or iOS
- Non-interactive script execution from RDM through Devolutions Gateway

Directions for a Future PowerShell SDK

Review PowerShell SDK distribution model

- Split client into smaller package that is excludes the entire PowerShell server
- Add option to include PowerShell application host into build output (Start-Job!)
- Enable reusing external PowerShell installations instead of shipping a full copy
- Make PowerShell SDK easier to rebuild from source for custom distributions

PSCONF.EU
Wiesbaden

PSRemoting in the Browser

@awakecoding.com

PSRemoting in the Browser — Javascript Stack

Original implementation was in Javascript

How it worked

- **Pure Javascript implementation of MS-PSRP** – expertise from ITManager.net acquisition
- **NTLM & Kerberos in the browser** via sspi-rs compiled to WASM with Javascript bindings
- **Devolutions Gateway** bridged WebSocket ↔ WinRM (HTTP inside a WSS tunnel)
- **JIT KDC proxy** in Gateway provided Kerberos line-of-sight alongside the WinRM connection
- **Interactive xterm.js terminal** to reconstruct a complete PowerShell console host

PSRemoting in the Browser — IronPosh (Rust)

Brand new MS-PSRP stack in Rust, compiled to WASM, ported to the browser

How it works

- **Pure Rust PSRemoting stack** compiled to WebAssembly
- **NTLM & Kerberos in the browser** via sspi-rs, without bindings (Rust to Rust)
- **Devolutions Gateway** bridges WebSocket ↔ WinRM (same as before, HTTP inside WSS)
- **JIT KDC proxy** in Gateway provides Kerberos line-of-sight alongside the WinRM connection
- **Interactive xterm.js terminal** to reconstruct a complete PowerShell console host

github.com/Devolutions/ironposh

PSRemoting in the Browser — Architecture

Browser		Devolutions Gateway		WinRM Server
IronPosh (Rust → WASM) PSRemoting client + sspi-rs NTLM / Kerberos in browser	**WebSocket (TLS)**	WebSocket ↔ WinRM bridge Terminates WS, forwards MS-PSRP frames to WinRM	**WinRM (HTTP/S)**	WinRM Service pwsh.exe session (MS-PSRP endpoint)
sspi-rs (in WASM) Requests Kerberos tickets for target SPN	**KDC Proxy (HTTPS)**	JIT KDC Proxy Forwards AS/TGS requests to domain KDC on demand	**Kerberos (TCP 88)**	Domain KDC (Active Directory) Issues Kerberos tickets

Devolutions Gateway provides line-of-sight to the KDC for the browser — no direct domain controller reachability required from the client

github.com/Devolutions/ironposh

Lessons Learned

The PowerShell SDK is a starting point, it's not flexible enough

- For advanced use cases like ours, you'll want to build your own stack

WinRM is very much alive, while we observe very little SSH adoption

- Why replace something that works very well in an Active Directory environment?

Local PSRemoting through subprocesses is extremely powerful

- An assembly load context? What's that? I have process isolation now!

We solved cross-platform Kerberos – use sspi-rs

- Don't build your own, it's a very complex stack, reuse ours instead

Future Development Ideas for Devolutions

Custom PowerShell SDK distribution

- Add missing pwsh.dll + apphost.exe as an option
- Integrate multi-pwsh as an alternative to apphost.exe
- Start patching things in the PowerShell core to meet our needs

Replace Win32-OpenSSH with pure .NET SSH client stack

- Expose Enter-RdmPSSession cmdlet built on top of RDM core
- Get rid of our custom Win32-OpenSSH distribution (less C libraries!)

Make gRPC a first-class PSRemoting transport type

- No choice now that it's been featured in Jeffrey Snover's presentation ☺

Q&A

15 minutes